# Running & Debugging Python in VS Code

Terminal execution, interpreter selection, launch.json, breakpoints and step-by-step debugging

**Purpose of this note**

Use this guide whenever you want to run a .py file from the VS Code terminal or inspect your program step by step with the Python debugger. It is designed for the Python_Engineering workspace used throughout the course.

## The mental model

```
WRITE CODE  ->  RUN FROM TERMINAL  ->  OBSERVE OUTPUT
                              |
                              v
                        SET BREAKPOINTS
                              |
                              v
                        START DEBUGGER
                              |
                              v
                INSPECT / STEP / FIX / RE-RUN
```

## Two different actions

| Step | What to do | Why |
|---|---|---|
| RUN | Execute the program normally. | Best when you simply want the final output or error message. |
| DEBUG | Execute under the debugger. | Best when you need to see how variables and program flow change line by line. |

**Important**

A simple Python file can be debugged in VS Code without a launch.json. Press F5 and choose Python File. We create launch.json because it saves a reusable, explicit configuration for the whole workspace.

# 1. Run a Python program from the terminal

Because you open Python_Engineering as your VS Code workspace, the integrated terminal normally starts at that workspace root. That makes the folder structure predictable and later simplifies imports, file paths and debugging.

## Open the terminal

| Step | What to do | Why |
|---|---|---|
| 1 | Open Python_Engineering in VS Code. | This is our workspace root. |
| 2 | Open Terminal > New Terminal or press Ctrl+`. | VS Code opens its integrated terminal. |
| 3 | Check the prompt/path. | It should point to your Python_Engineering folder. |

## Verify Python

On Windows, first try:

```
python --version
```

If the command is not recognized, try the Python launcher:

```
py --version
```

## Run the current exercise

Option A - navigate into the exercises folder:

```
cd .\01_python_bootcamp\week_01_logic\class_01_equation_to_algorithm\exercises\npython ex01_dynamic_pressure.py
```

Option B - stay at the workspace root and run the path directly:

```
python .\01_python_bootcamp\week_01_logic\class_01_equation_to_algorithm\exercises\ex01_dynamic_pressure.py
```

**Recommended while learning**

Use Option A often at first. Navigating with cd helps you understand where your program is running from. Later, when projects become modular, understanding the working directory will be essential.

## Useful terminal commands on Windows / PowerShell

| Step | What to do | Why |
|---|---|---|
| pwd | Show the current folder. | Confirms your working directory. |
| ls / dir | List files and folders. | Lets you inspect what is available. |

| Step | What to do | Why |
| --- | --- | --- |
| cd folder | Enter a folder. | Changes working directory. |
| cd .. | Go one level up. | Useful when navigating the project tree. |
| cls | Clear the terminal. | Keeps the console readable. |

# 2. Make sure VS Code uses the correct Python interpreter

The terminal and debugger should use the Python environment you intend. This matters more and more once we install NumPy, SciPy, pandas or project-specific dependencies.

## Select the interpreter

| Step | What to do | Why |
|---|---|---|
| 1 | Press Ctrl+Shift+P. | Opens the Command Palette. |
| 2 | Search for Python: Select Interpreter. | Shows Python environments detected by VS Code. |
| 3 | Select the interpreter/environment for this course. | VS Code associates it with the workspace. |
| 4 | Open a fresh terminal if needed. | Ensures activation is refreshed. |

**Diagnostic check**

Run python --version and later, when we use packages, python -m pip list. If the debugger behaves differently from the terminal, checking the selected interpreter is one of the first troubleshooting steps.

## Running without typing the command

The Python extension also provides Run Python File in Terminal in the top-right of the editor. It activates the selected interpreter and runs the current file. This is convenient, but you should still learn the terminal command because it makes execution explicit.

```
python your_script.py
```

## Run vs. REPL

Do not confuse running a file with entering the Python REPL. If you type only python, you normally enter an interactive session identified by the >>> prompt. To execute a saved script, include the file name.

```
python
>>> 2 + 2
4
>>> exit()

python ex01_dynamic_pressure.py
```

# 3. Create launch.json for Python debugging

VS Code stores debugger configurations in a file named launch.json inside a .vscode folder in the workspace. For our course, the expected location is:

```
Python_Engineering\n└── .vscode\n    └── launch.json
```

## Generate it from VS Code

| Step | What to do | Why |
|---|---|---|
| 1 | Open the Run and Debug view with Ctrl+Shift+D. | This is the main debugger panel. |
| 2 | Choose create a launch.json file (or Run > Open Configurations). | VS Code starts configuration generation. |
| 3 | Choose Python Debugger. | Selects the current Python debugger implementation. |
| 4 | Choose Python File. | Creates a configuration for a normal Python script. |
| 5 | Save the generated file. | The configuration is now reusable from the workspace. |

## Recommended configuration for our course

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python Debugger: Current File",
            "type": "debugpy",
            "request": "launch",
            "program": "${file}",
            "console": "integratedTerminal",
            "cwd": "${workspaceFolder}"
        }
    ]
}
```

**Do not use the old debugger type**

Current VS Code Python debugging uses "type": "debugpy". If you find an old tutorial using "type": "python", update it.

# 4. Understand what launch.json is saying

Do not treat configuration files as magic. Read them as instructions given to VS Code.

| Step | What to do | Why |
|---|---|---|
| version | Schema version for the launch configuration. | Usually leave it unchanged. |
| name | Human-readable name shown in Run and Debug. | You can rename it. |
| type | Debugger implementation: debugpy. | Required for current Python debugging. |
| request | launch means start a new Python process. | Later we may also use attach. |
| program | ${file} means the currently open file. | Perfect for our exercise scripts. |
| console | integratedTerminal sends I/O to VS Code terminal. | Useful for print() and input(). |
| cwd | ${workspaceFolder} fixes the working directory at Python_Engineering. | Makes relative paths predictable. |

## Why cwd matters

Suppose a future program reads data/input.csv. Relative paths are interpreted from the current working directory. By setting the workspace as cwd, we make program behavior more reproducible.

```
"cwd": "${workspaceFolder}"
```

## Adding command-line arguments later

When we build scripts that accept input parameters, launch.json can pass them automatically:

```
"args": [
    "--altitude", "10000",
    "--mach", "0.8"
]
```

You do not need this yet. It is here to show why a saved debugger configuration becomes useful as projects grow.

# 5. Debug step by step

Debugging is not only for fixing crashes. It is one of the best tools for learning program logic because you can watch the state of the algorithm evolve.

## Example program

```
rho = 1.225
velocity = 100.0

dynamic_pressure = 0.5 * rho * velocity**2

print(dynamic_pressure)
```

## Set a breakpoint

Click in the gutter to the left of a line number. A red dot appears. When execution reaches that line, VS Code pauses before executing it.

## Start debugging

- F5 - Continue / Start Debugging: run until the next breakpoint or the end.
- F10 - Step Over: execute the current line and move to the next one without entering called functions.
- F11 - Step Into: enter a function call so you can inspect its internal execution.
- Shift+F11 - Step Out: finish the current function and return to its caller.
- Shift+F5 - Stop: terminate the debugging session.

## What to inspect

| Step | What to do | Why |
|---|---|---|
| Variables | Current local and global values. | See how state changes after each line. |
| Watch | Expressions you choose to monitor. | Useful for formulas such as b**2 - 4*a*c. |
| Call Stack | Which functions led to the current line. | Becomes crucial when our programs become modular. |
| Debug Console | Evaluate expressions while paused. | Test values without modifying the script. |

# 6. Suggested workflow for your Class 01 exercises

For the first four exercises, use the debugger deliberately. The goal is to connect mathematical reasoning with changing program state.

| Step | What to do | Why |
|---|---|---|
| 1 | Write the INPUTS / OUTPUTS / EQUATIONS / SPECIAL CASES / ALGORITHM comments. | Think before coding. |
| 2 | Write the Python implementation. | Translate the algorithm. |
| 3 | Run it from the terminal. | Check normal behavior. |
| 4 | Add a breakpoint before the main equation or first if statement. | Choose a meaningful observation point. |
| 5 | Press F5 and step with F10. | Watch execution order. |
| 6 | Inspect variables after each important line. | Connect code to the mathematical state. |
| 7 | Try a deliberately unusual input. | Learn how edge cases flow through the program. |

## Example: quadratic equation

Place a breakpoint immediately after computing the discriminant:

```
discriminant = b**2 - 4*a*c

if discriminant > 0:
    ...
elif discriminant == 0:
    ...
else:
    ...
```

Then test three input sets so the debugger follows all three branches: one with two real roots, one repeated root, and one with no real roots.

## Common problems

| Step | What to do | Why |
|---|---|---|
| python is not recognized | Python is not on PATH or launcher differs. | Try py --version, verify installation and VS Code interpreter. |

| Step | What to do | Why |
| --- | --- | --- |
| Wrong packages/interpreter | Terminal and debugger are using another environment. | Use Python: Select Interpreter. |
| Breakpoint is ignored | The line may not be executable or the wrong file/configuration is running. | Check the active file and selected debug configuration. |
| launch.json error | Old examples may use type: python. | Use type: debugpy. |
| File not found | Your working directory differs from what your relative path assumes. | Inspect pwd and use cwd: ${workspaceFolder} when appropriate. |

# 7. One-page cheat sheet

## Run from terminal

```
# Open terminal
Ctrl+`

# Check Python
python --version

# Navigate
cd .\01_python_bootcamp\week_01_logic\class_01_equation_to_algorithm\exercises

# Run a script
python ex01_dynamic_pressure.py
```

## Create debugger configuration

```
Ctrl+Shift+D
-> create a launch.json file
-> Python Debugger
-> Python File
```

## Minimal launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal",
      "cwd": "${workspaceFolder}"
    }
  ]
}
```

## Debug keys

| Step | What to do | Why |
|---|---|---|
| F5 | Start / Continue | Run until breakpoint or finish. |
| F10 | Step Over | Execute current line. |
| F11 | Step Into | Enter a function. |
| Shift +F11 | Step Out | Leave current function. |
| Shift +F5 | Stop | End debugging. |

**Course habit**

When a result surprises you, do not immediately add print() everywhere. First ask: At which line does the program state stop matching my mathematical expectation? Place a breakpoint there and inspect the state. That is the debugging mindset we will build throughout the course.

---

Reference basis: official Visual Studio Code documentation for the integrated terminal, running Python code, and Python debugging/debugpy (checked August 2026).